

iOS 中 4 个实用 UI 工具库详解

iOS 项目中 3 个实用 UI 工具库的使用详解

一、导航控制器增强：FDFullscreenPopGesture

1. 核心功能与价值

FDFullscreenPopGesture 是一款轻量级导航控制器增强库，核心作用是 **为 App 添加全屏返回手势**，解决 iOS 原生返回手势的体验局限：

- 原生局限：仅支持屏幕左侧边缘滑动返回，操作范围窄；
- 库优势：支持从屏幕任意位置向右滑动返回，符合微信、淘宝等主流 App 交互习惯；
- 集成成本：一行代码无需额外配置，通过运行时机制自动生效。

2. 核心实现原理

库内部通过 **Objective-C 运行时（Method Swizzling）** 为 UINavigationController 注入全屏手势逻辑，核心代码思路如下：

```
- (void)fd_setupFullscreenPopGesture {  
    // 1. 创建全屏滑动手势（Pan手势）  
    UIPanGestureRecognizer *fullscreenPanGesture = [[UIPanGestureRecognizer alloc]  
                                                    initWithTarget:self  
                                                    action:@selector(fd_handleNavigationTransition:)];  
  
    // 2. 设置手势代理，处理与其他手势（如TableView滑动）的冲突  
    fullscreenPanGesture.delegate = self;  
    fullscreenPanGesture.maximumNumberOfTouches = 1; // 仅支持单指滑动  
  
    // 3. 将手势添加到导航控制器的根视图，确保全屏可触发  
    [self.view addGestureRecognizer:fullscreenPanGesture];  
  
    // 4. 禁用原生边缘返回手势（避免与全屏手势冲突）
```

```
self.interactivePopGestureRecognizer.enabled = NO;
}
// 手势触发后的过渡逻辑（内部实现页面返回动画）
- (void)fd_handleNavigationTransition:(UIPanGestureRecognizer *)gesture {
    // 计算滑动距离、速度，判断是否触发返回
    // 内部调用导航控制器的 popViewControllerAnimated: 方法完成返回
    // ...（省略具体动画与判断逻辑）
}
```

3. 控制器级别的个性化配置

支持在单个控制器中灵活开关全屏手势或自定义导航栏行为，满足特殊页面需求：

```
// 在需要配置的 ViewController.m 中设置以下属性（库提供的分类属性）
- (void)viewDidLoad {
    [super viewDidLoad];

    // 1. 禁用当前页面的全屏返回手势（如支付页、登录页）
    self.fd_interactivePopDisabled = YES;

    // 2. 强制当前页面隐藏导航栏（无需在 navigationItem 中设置）
    self.fd_prefersNavigationBarHidden = YES;

    // 3. 返回手势触发时，隐藏导航栏背景（仅保留按钮，实现渐变效果）
    self.fd_prefersNavigationBarBackgroundHidden = YES;
}
```

二、导航栏样式统一管理：WRNavigationBar

1. 核心功能与价值

WRNavigationBar 是一款专注于 **导航栏样式统一管理** 的库，解决原生导航栏配置繁琐、页面切换样式混乱的问题：

- 原生局限：不同页面切换时，导航栏背景色、标题色需重复设置，易出现闪烁或样式不一致；
- 库优势：提供链式调用 API，支持全局默认配置 + 单个页面个性化调整，支持渐变、透明度动画等高级效果；

- 适用场景：App 内多页面导航栏样式统一，或需要复杂视觉效果（如滚动时导航栏渐变）。

2. 全局默认配置（AppDelegate 中）

在应用启动时配置全局导航栏默认样式，减少重复代码：

```
#import "WRNavigationBar.h"
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(
NSDictionary *)launchOptions {
    // 1. 启用 WRNavigationBar 全局管理（必须调用）
    [WRNavigationBar wr_widely];

    // 2. 设置黑名单（指定不使用 WRNavigationBar 的控制器，如 WebView 页面）
    [WRNavigationBar wr_setBlacklist:@[
        [ZYWebBaseViewController class],
        [SomeSpecialViewController class]
    ]];

    // 3. 配置全局默认样式
    [WRNavigationBar wr_setDefaultNavBarBarTintColor:[UIColor whiteColor]]; // 导航栏背景色
    [WRNavigationBar wr_setDefaultNavBarTitleColor:[UIColor blackColor]]; // 标题文字颜色
    [WRNavigationBar wr_setDefaultNavBarTintColor:[UIColor blackColor]]; // 按钮文字/图标
    颜色
    [WRNavigationBar wr_setDefaultNavBarShadowImageHidden:NO]; // 显示导航栏底部
    阴影（默认显示）

    return YES;
}
```

3. 单个控制器个性化配置

在具体页面中调整导航栏样式，覆盖全局默认配置，通常在 `viewWillAppear` 中设置：

```
- (void)viewWillAppear:(BOOL)animated {
    [super viewWillAppear:animated];

    // 1. 设置导航栏背景色（红色）
    [self wr_setNavBarBarTintColor:[UIColor redColor]];
```

```
// 2. 设置标题文字颜色（白色）
[self wr_setNavBarTitleColor:[UIColor whiteColor]];

// 3. 设置导航栏背景透明度（0.8，半透明效果）
[self wr_setNavBarBackgroundAlpha:0.8];

// 4. 隐藏导航栏底部阴影（避免与页面内容重叠）
[self wr_setNavBarShadowImageHidden:YES];

// 5. （可选）设置导航栏背景渐变（从红色到橙色）
[self wr_setNavBarGradientColors:@[
    [UIColor redColor],
    [UIColor orangeColor]
]];
}
// （可选）页面消失时恢复全局默认样式，避免影响后续页面
- (void)viewWillDisappear:(BOOL)animated {
    [super viewWillDisappear:animated];
    [self wr_resetNavBar]; // 恢复全局默认样式
}
```

三、自定义弹窗与底部菜单：LEEAlert & MHActionSheet

1. LEEAlert - 多功能自定义弹窗

LEEAlert 是一款支持多样式的弹窗库，可实现确认弹窗、输入框弹窗、底部弹窗等，替代系统 UIAlertController，支持高度自定义。

（1）基础确认弹窗（如清除缓存提示）

```
// 链式调用配置弹窗，无需手动初始化
[LEEAlert alert].config
    .LeeTitle(@"确定清除本地缓存？") // 弹窗标题
    .LeeTitleFont([UIFont boldSystemFontOfSize:16]) // 标题字体
    .LeeContent(@"清除后缓存数据将无法恢复，请确认") // 弹窗内容
    .LeeContentFont([UIFont systemFontOfSize:14]) // 内容字体
```

```
.LeeCancelAction(@"取消", ^(UIButton * _Nonnull btn) {
    // 取消按钮点击事件（空实现可传 nil）
})
.LeeAction(@"确定", ^(UIButton * _Nonnull btn) {
    // 确定按钮点击事件：执行清除缓存
    [self clearLocalCache];
})
.LeeShow(); // 显示弹窗
```

(2) 带输入框的弹窗（如编辑昵称）

```
// 用 __block 变量保存输入框引用，便于后续获取输入内容
__block UITextField *nicknameTextField = nil;
[LEEAlert alert].config
.LeeTitle(@"编辑昵称")
.LeeContent(nil) // 无内容，仅显示输入框
.LeeAddTextField:^(UITextField * _Nonnull textField) {
    // 配置输入框属性
    textField.placeholder = @"请输入新昵称（1-10个字符）";
    textField.textColor = [UIColor darkGrayColor];
    textField.borderStyle = UITextBorderStyleRoundedRect;
    textField.returnKeyType = UIReturnKeyDone;

    // 保存输入框引用
    nicknameTextField = textField;
})
.LeeCancelAction(@"取消", nil)
.LeeAction(@"确定", ^(UIButton * _Nonnull btn) {
    // 获取输入内容并校验
    NSString *newNickname = [nicknameTextField.text stringByTrimmingCharactersInSet:[
        NSCharacterSet whitespaceCharacterSet]];
    if (newNickname.length > 0 && newNickname.length <= 10) {
        // 校验通过，更新昵称
        self.nicknameLabel.text = newNickname;
    } else {
        // 校验失败，提示用户
        [MBProgressHUD showError:@"请输入1-10个字符的昵称"];
    }
}
```

```
}  
.LeeShow();
```

2. MHActionSheet - 底部弹出菜单

MHActionSheet 是一款轻量级底部弹出菜单库，替代系统 UIActionSheet，支持自定义选项样式、图标等，项目中虽无明确使用示例，但根据库设计规范，基础使用方式如下：

(1) 基础使用流程

```
// 1. 初始化底部菜单  
MHActionSheet *actionSheet = [[MHActionSheet alloc] init];  
actionSheet.delegate = self; // 设置代理，处理点击事件  
actionSheet.title = @"请选择操作"; // 菜单标题（可选）  
// 2. 添加选项（支持文字、图标，具体方法需参考库版本）  
// 示例：添加“分享”“保存图片”选项  
[actionSheet addItemWithTitle:@"分享到微信"  
    image:[UIImage imageNamed:@"icon_wechat"]  
    handler:^(NSInteger index) {  
    // 选项点击事件（也可通过代理方法处理）  
    [self shareToWeChat];  
}];  
[actionSheet addItemWithTitle:@"保存图片"  
    image:[UIImage imageNamed:@"icon_save"]  
    handler:^(NSInteger index) {  
    [self saveImageToAlbum];  
}];  
// 3. 添加取消选项（默认在底部，样式与其他选项区分）  
[actionSheet addCancelItemWithTitle:@"取消"  
    handler:nil]; // 取消按钮点击事件（空实现）  
// 4. 显示底部菜单  
[actionSheet showInView:self.view];
```

(2) 代理方法处理点击事件（可选）

```
// 实现 MHActionSheetDelegate 代理方法  
- (void)actionSheet:(MHActionSheet *)actionSheet didSelectItemAtIndex:(NSInteger)index
```

```
title:(NSString *)title {
    // 根据选项索引处理不同逻辑
    switch (index) {
        case 0:
            [self shareToWeChat];
            break;
        case 1:
            [self saveImageToAlbum];
            break;
        default:
            break;
    }
}
```

总结

库名称	核心功能	优势与适用场景	关键特性
FDFullscreenPopGesture	全屏返回手势	一行代码集成，提升导航体验，适用于所有导航页面	支持控制器级开关，无手势冲突
WRNavigationBar	导航栏样式统一管理	链式 API，全局 + 个性化配置，支持渐变 / 透明度	避免样式闪烁，降低重复代码
LEEAlert	多功能自定义弹窗	支持输入框 / 渐变 / 动画，替代系统弹窗，高度自定义	链式调用，无需复杂初始化
MHActionSheet	底部弹出菜单	轻量，支持图标 / 文字，替代系统 UIActionSheet	代理 + block 双方式处理点击，样式可定制

这 4 个库均为 iOS 开发主流工具，核心价值是“降低复杂 UI 功能的实现成本”，通过简洁 API 让开发者聚焦业务逻辑，同时保证交互体验与视觉一致性。

（注：文档部分内容可能由 AI 生成）